Power-Grid Plugin Building Guide

Version B | January 2013

Content

Introduction 3

Building a Plugin 3

- Namespace 3
- Controls 4
- ControlSets 4
- Skins 6
  - Layout Elements 7
  - Layout Elements (Type: "Skin") 8
  - Layout Elements (Type: "Text" / Type: "Input") 9
  - Layout Elements (Type: "Gfx") 10
- Gfxs 11
  - Layering 11
  - Tiling 12
- Scripts 12
- Dynamic Libraries 13

Sample Plugin 14

Appendix A: Colors 16
Hexadecimal notation 16
Named colors 16
Appendix B: Alignment 17
Appendix C: Config Options 17
Type: None (hidden) 18
Type: Label (non-editable text) 18
Type: Text (textbox) 18
Type: Password (textbox) 18
Type: Check (a single checkbox) 19
Type: Radio (set of radio buttons) 19
Type: File (file selector / open file dialog) 19
Type: Folder (folder selector / choose folder dialog) 19
Type: Shortcut 20
Type: Drop (dropdown) 20
Type: Button 21
Type: Spinner 21
Type: Group (group of ConfigOptions) 21
Type: SpinDrop 22
Type: TextList 22
Type: Align 22
Type: Position 22
Type: ColorPicker 22

Introduction

Plugins in the Roccat Power-Grid may introduce new Functionality, Gfxs, Controls, Grids and Data Sources and make them available to the user and/or other plugins.

Plugins are stored in subdirectories of "%USERPROFILE%\Documents\ROCCAT\Power-Grid\Plugins"

Building a Plugin

Namespace

Every Plugin must provide a "plugin.json" file in its root folder with the following content:

plugin.json

```
{
  Name:"MyPlugin",
  Includes:[],
  Controls:[],
  CtrlSets:[],
  Skins:[],
  Gfxs:[],
  Scripts:[],
  DynamicLibraries:[]
}
```

The "Name" property defines the plugin's name and namespace. All contents of this plugin will reside inside this namespace. Plugins that don't specify a Name won't be loaded.

"Includes" is an array of relative file paths pointing to other "*.json" files in the same format as "plugin.json" containing additional contents.

The other properties are optional arrays used to define the contents of the plugin as described in the following in more detail.

Controls

Custom controls can be defined directly in plugin.json in the Controls-array.

Controls have the following properties:

Property (Type)	Description	Example
Name (String)	Name of the Control. The Control will be accessible through the Identifier String "Namespace:Name".	`Name:"MyControl",`
Skins (Array of Strings)	An array of Skin identifiers listing all Skins available for the Control.	`Skins:[ "MyPlugin:MySkin1", "MyPlugin:MySkin2", ],`
Functions (Array of Strings)	An array of Function identifiers listing all Functions used by the Control.	`Functions:[ "Function1", "Function2", ],`
SkinIdx (Integer)	Index of the Skin to be used. (Beginning at index 1)	`SkinIdx:1,`
Width (Integer)	Width of the Control in tiles.	`Width:1,`
Height (Integer)	Height of the Control in tiles.	`Height:2,`
Config (Complex)	Configuration options and description of the control. (For details on Configuration Option JSON Objects see Appendix C: Config Options)	`Config:{ Description:"desc", Options:[] },`
ExtParams (Complex)	A JSON Object that is passed to the Control's JavaScripts as is. (Optional; Advanced users)	`ExtParams:{foo:"bar"},`

ControlSets

In order to make custom controls accessible to the user you have to add them to a ControlSet.

ControlSets are defined in separate files. The relative file paths of the ControlSet files are then listed in the "CtrlSets" array in "plugin.json" like this:

```
CtrlSets:[
  "Sample.ctrlset.json"
],
```

The ControlSet JSON-file has the following properties:

Property (Type)	Description	Example
Type (String)	Type definition of the file. Always "ControlSet".	`Type:"ControlSet",`
Name (String)	Display name of the ControlSet as to be shown in the ControlSets dropdown menu.	`Name:"[Plugin] Sample",`
ExpandCategories (Array of Strings)	List of Categories that should be expanded by default or "*" for all.	`ExpandCategories:["*"],`
Elements (Array of Complex)	An array of JSON Objects resembling the Controls that are contained in this ControlSet.	`Elements:[],`

The JSON Objects resembling the individual Controls have the following properties:

Property (Type)	Description	Example
Name (String)	Display name of the Control as to be shown in the ControlSet.	`Name:"My own Control",`
Control (String)	The Identifier String of the Control to be used.	`Control: "MyPlugin:MyControl",`
Category (String)	Name of the Category the Control should reside in.	`Category:"My Category",`

A ControlSet file may look like this:

Sample.ctrlset.json

```
{
  Type:"ControlSet",
  Name:"[Plugin] Sample",
  ExpandCategories:["*"],
  Elements:[
    {
      Name:"My own Control",
      Control:"MyPlugin:MyControl",
      Category:"My Category"
    },
    {
      Name:"My other Control's Name",
      Control:"MyPlugin:MyOtherControl",
      Category:"My Category"
    }
  ]
}
```

Skins

Skins define how a Control's input and output works and should look like.

Property (Type)	Description	Example
Name (String)	Name of the Skin. The Skin will be accessible through the Identifier String "Namespace:Name".	`Name:"MySkin",`
OnInit (String)	Identifier String of the function to be called before the Skin is initialized.	`OnInit: "MyPlugin:MyFunc",`
OnPostInit (String)	Identifier String of the function to be called after the Skin has been initialized.	`OnPostInit: "MyPlugin:MyFunc",`
OnTick (String)	Identifier String of the function to be called every time the Skin is going to be redrawn.	`OnTick: "MyPlugin:MyFunc",`
OnUpdate (String)	Identifier String of the function to be called	`OnUpdate: "MyPlugin:MyFunc",`
OnMousePress (String)	Identifier String of the function to be called when a mouse button is pushed (finger is placed on the touchscreen) within the borders of the Skin.	`OnMousePress: "MyPlugin:MyFunc",`
OnMouseMove (String)	Identifier String of the function to be called when the mouse is moved.	`OnMouseMove: "MyPlugin:MyFunc",`
OnMouseRelease (String)	Identifier String of the function to be called when the mouse button is released (finger leaves touchscreen) after an "OnMousePress" event.	`OnMouseRelease: "MyPlugin:MyFunc",`
OnMouseClick (String)	Identifier String of the function to be called when a mouse button is pressed and released (finger is placed and removed from the touchscreen) within the borders of the Skin.	`OnMouseClick: "MyPlugin:MyFunc",`
OnMouseWheel (String)	Identifier String of the function to be called when the mouse wheel is scrolled.	`OnMouseWheel: "MyPlugin:MyFunc",`
Width (Integer)	Width of the Skin in number of tiles.	`Width:2,`
Height (Integer)	Height of the Skin in number of tiles.	`Height:4,`
Layouts (Array of Complex)	An array of Layout elements that describe how this Skin is to be drawn.	`Layouts:[],`

Layout Elements

Layout elements describe where and how text labels, input fields, GFXs should be displayed or where other generalized Skins should be placed with what actual dimensions and labels.

LEs (Layout elements) have the following properties:

Property (Type)	Description	Example
Name (String)	Name of the LE. The LE will be accessible through the Identifier String "Namespace:Name".	`Name:"MySkin",`
OnInit (String)	Identifier String of the Function to be called before the LE is initialized.	`OnInit: "MyPlugin:MyFunc",`
OnPostInit (String)	Identifier String of the Function to be called before the LE has been initialized.	`OnPostInit: "MyPlugin:MyFunc",`
OnTick (String)	Identifier String of the Function to be called on every application tick.	`OnTick: "MyPlugin:MyFunc",`
OnUpdate (String)	Identifier String of the Function to be called every time the LE is going to be redrawn.	`OnUpdate: "MyPlugin:MyFunc",`
OnMousePress (String)	Identifier String of the Function to be called when a mouse button is pushed (finger is placed on the touchscreen) within the borders of the LE.	`OnMousePress: "MyPlugin:MyFunc",`
OnMouseMove (String)	Identifier String of the Function to be called when the mouse is moved.	`OnMouseMove: "MyPlugin:MyFunc",`
OnMouseRelease (String)	Identifier String of the Function to be called when the mouse button is released (finger leaves touchscreen) after an "OnMousePress" event.	`OnMouseRelease: "MyPlugin:MyFunc",`
OnMouseClick (String)	Identifier String of the Function to be called when a mouse button is pressed and released (finger is placed and removed from the touchscreen) within the borders of the LE.	`OnMouseClick: "MyPlugin:MyFunc",`
OnMouseWheel (String)	Identifier String of the Function to be called when the mouse wheel is scrolled.	`OnMouseWheel: "MyPlugin:MyFunc",`
MouseEnabled (Boolean)	Explicitly enables or disables mouse events. (Optional)	`MouseEnabled:true,`
MouseMask (String)	Identifier String of the Gfx to be used to decide which pixels inside the LE's bounds are clickable. A value of"None" means all pixels are clickable (Optional; defaults to the value of the "Gfx" property if it is provided or "none" otherwise)	`MouseMask:"None",`
MouseEvtMinAlpha (Integer)	Minimum Alpha-channel value needed for a pixel in MouseMask to consider it clickable. (Optional)	`MouseEvtMinAlpha:127,`

Property (Type)	Description	Example
Visible (Boolean)	Controls the visibility of the LE.	`Visible:true,`
Left (String)	Distance from the left border of the surrounding container specified either in pixels (Integer: 100) or fractions of tiles (Float: 1.5). (Optional)	`Left:100,`
Top (String)	Distance from the top border of the surrounding container specified either in pixels (Integer: 100) or fractions of tiles (Float: 1.5). (Optional)	`Top:20,`
Width (String)	Width of the LE in pixels (Integer: 100) or fractions of tiles (Float: 1.5).	`Width:1.0,`
Height (String)	Height of the LE in pixels (Integer: 100) or fractions of tiles (Float: 1.5).	`Height:2.0,`
Align (String)	A String defining the alignment of the LE inside the parent container. See Appendix B: Alignment for details. (Only used when Left and Top are not specified)	`Align:"CenterBottom",`
Opacity (Float)	Opacity of the LE. (Optional)	`Opacity:0.5,`
ExtendHitTest (Integer)	Number of pixels to extend the hit/click box of the LE by in all directions. (Optional)	`ExtendHitTest:10,`
ExtParams (Complex)	A JSON Object that is passed to the Control's JavaScripts as is. (Optional)	`ExtParams:{foo:"bar"},`
Type (String)	One of the following Strings defining the Layout element's type: "Text", "Input", "Gfx" and "Skin".	`Type:"Text",`

Depending on the Layout element type the following properties also become available:

Layout Elements (Type: "Skin")

Property (Type)	Description	Example
Skin (String)	Identifier String of the Skin to be used as template for this LE.	`Skin: "MyPlugin:MyButton",`
Scrollable (Boolean)	Specifies whether this LE should be scrollable when content overflows. (Optional; defaults to false)	`Scrollable:true,`
Override (Complex)	Structure used to override properties of the used Skin's LEs. Currently only the "Text"-property can be overridden. (Optional)	`Override:{ MyTextTypeLE:{ Text:"Send" } },`

Layout Elements (Type: "Text" / Type: "Input")

"Text"-type Layout elements are used to display text on a skin.

Property (Type)	Description	Example
Color (String)	A String specifying the Color in which the Text should be drawn by default. See Appendix A: Colors for details on supported formats.	`Color:"White",`
ColorP (String)	A String specifying the Color in which the Text should be drawn when clicked. See Appendix A: Colors for details on supported formats.	`Color:"Red",`
Text (String)	The Text to be shown in this LE.	`Text:"Some blah...",`
Font (String)	Font identifier specifying the font and optionally the size of the font to use for rendering the text. The currently available Font identifiers are "EuroD", "EuroDO", "EuroB" and "EuroBO".	`Font:"EuroD,12",`
Toggle (Boolean)	Set to true to make the LE stay in the "pressed"-state after being clicked and revert to "normal"-state only after a second click instead of resetting immediately after mouse release.	`Toggle:false,`
WordWrap (Boolean)	Insert automatic newlines when the text would overflow.	`WordWrap:true,`
TextStroke (Boolean)	Apply a strikethrough effect on the text.	`TextStroke:false,`
TextUnderline (Boolean)	Apply an underline effect on the text.	`TextUnderline:true,`
TextAlign (String)	A String defining the alignment of the Text inside the container specified by Width and Height properties. See Appendix B: Alignment for details.	`TextAlign:"LeftTop",`

Layout Elements (Type: "Gfx")

Property (Type)	Description	Example
Color (String)	A String identifying the Color in which the Text should be drawn by default. See Appendix A: Colors for details on supported formats.	`Color:"White",`
ColorP (String)	A String identifying the Color in which the Text should be drawn when clicked. See Appendix A: Colors for details on supported formats.	`Color:"Red",`
Gfx (String)	Identifier String of the Gfx to be drawn in "normal"-state.	`Gfx:"MyPlugin:MyGfx",`
GfxA (String)	Identifier String of the Gfx to be drawn in "active"-state. (DCLayout::active in JS)	`GfxA:"MyPlugin:MyGfx",`
GfxP (String)	Identifier String of the Gfx to be drawn in "pressed"-state. (mouse button is pressed)	`GfxP:"MyPlugin:MyGfx",`
GfxM (String)	Identifier String of the Gfx to be used as Alpha-channel for the LE. (Optional)	`GfxM:"MyPlugin:MyGfx",`
Toggle (Boolean)	Set to true to make the LE stay in the "pressed"-state after being clicked and revert to "normal"-state only after a second click instead of resetting immediately after mouse release.	`Toggle:false,`
PressedOvl (Boolean)	If true GfxP is drawn on top of Gfx instead of replacing it when in "pressed"-state.	`PressedOvl:false,`

Mask (Complex)	Only draws the portion of the area between Start and End that is specified by the Input. Supported Types are "Horizontal", "Vertical" and "Radial". (Optional)	`Mask:{`
		`  Type:"Horizontal",`
		`  Start:0.0,`
		`  End:1.0,`
		`  Input:0`
		`},`

Mapping (String)	Maps the LE's Inputs to the Control's Functions and their Outputs. The Format is "Input->Function.Output". (Optional)	`Mapping:` `  "0->0.0,1->1.0"`

Gfxs

Gfxs import image files into the plugin's Namespace, so they can be referred to in Skins.

Property (Type)	Description	Example
Name (String)	Name of the Gfx. The Gfx will be accessible through the Identifier String "Namespace:Name".	`Name:"MyGfx",`
Rsc (String)	The Filename where the Gfx should be loaded from.	`Rsc:"img/sample.png",`
Tiling (String)	One of the following Strings identifying the tiling mode: "None", "1x1", "3x1", "1x3", "3x3", "Scale".	`Tiling:"3x1",`

Tiles
(Array of Strings)
	A list of Strings defining the position and dimensions of the individual tiles.
	The structure of the Strings is: "Left,Top,Width,Height,RepeatXY"
	RepeatXY is an optional two character string defining whether the tile is repeated or scaled horizontally and vertically.
		`Tiles:[`
		`  "0,0,40,160,rs",`
		`  "40,0,20,160,sr",`
		`  "60,0,40,160,rs"`
		`],`

Layers
(Array of Complex)
	An array of Layer JSON Objects describing the individual layers this Gfx consists of.	`Layers:[`
		`  {}`
		`],`

Layering

Layering is used to compose a layered Gfx consisting of multiple individual Gfxs drawn on top of each other.

The Layer JSON Objects used to describe the individual layers contain the following properties:

Property (Type)	Description	Example
Gfx (String)	Identifier String of the Gfx to use for this Layer.	`Gfx:"MyPlugin:MyGfx",`
Align (String)	A String defining the alignment of the Layer-Gfx inside the parent container. See Appendix B: Alignment for details. (Only used when Left and Top are not specified)	`Align:"LeftTop",`
Left (String)	Distance from the left border of the surrounding container specified either in pixels (Integer: 100) or fractions of tiles (Float: 1.5). (Optional)	`Left:"0.0",`
Top (String)	Distance from the top border of the surrounding container specified either in pixels (Integer: 100) or fractions of tiles (Float: 1.5). (Optional)	`Top:"100",`
Width (String)	Width of the Layer-Gfx in pixels (Integer: 100) or fractions of tiles (Float: 1.5).	`Width:"300",`
Height (String)	Height of the Layer-Gfx in pixels (Integer: 100) or fractions of tiles (Float: 1.5).	`Height:"1.8",`

Tiling

Tiling can be used to create Gfxs with variable sizes.

Be advised that if both features, layering and tiling, are combined the tiling is applied to the Gfx resulting from the layering.

Depending on the tiling mode specified in the Tiling-property certain portions of an image will be scaled in order to make the resulting Gfx fit the sizes of the individual Layout Elements it gets assigned to.

Scripts

Scripts are used to implement the actual functionality for Controls. This section is used to import these scripts into the Plugin's Namespace in order to make specified JavaScript-Functions available for use in Skins.

Property (Type)	Description	Example
Name (String)	Name of the Script. The Script will be accessible through the Identifier String "Namespace:Name".	`Name:"MyScript",`
Rsc (String)	The Filename where the Script should be loaded from.	`Rsc:"src/sample.qs",`

Functions
(Array of Strings)
	An array of function names that lists functions defined in the Rsc-File that will be made available to Skins and other Plugins.	`Functions:[`
		`  "ButtonOnInit",`
		`  "ButtonOnTick",`
		`],`

For details on what API is available to the JavaScript-Engine see the **Power-Grid JavaScript Reference Manual**.

Dynamic Libraries

Dynamic Libraries are needed to implement additional functions that enable communication with the host system and the network. This section is used to specify dynamic libraries that should be loaded and made available in the plugin's namespace for use in JavaScript.

Just list the filenames of the dynamic libraries shipped with your plugin in this section in order to make them get loaded on startup.

Prerequisites for creating Dynamic Libraries

- Qt Libraries version 4.8.3
  http://qt-project.org/downloads#qt-lib
- The MSVC 10 SP1 Toolchain (As provided in Visual C++ 2010 SP1)
  http://www.microsoft.com/visualstudio/eng/products/visual-studio-2010-express
- QtCreator 2.5.2
  http://qt-project.org/downloads#qt-creator

Dynamic Libraries need to include RoccatPowerGrid.h and link against RoccatPowerGrid.lib.

Power-Grid loads these using QPluginLoader. A dynamic library provides either an IFuncProvider or an IIncomingDataSource. Use the Q_INTERFACES() macro for class declaration and Q_EXPORT_PLUGIN2() macro for implementation.

See also: http://doc.qt.digia.com/qt/plugins-howto.html

Additional information can be found in Sample2 which implements a simple dynamic library plugin.

For details on what API is available to Dynamic Libraries see the **Power-Grid Dynamic Library Reference Manual**.

Sample Plugin

Now let's take a look at a pretty basic sample plugin.

The Plugin's name is "Sample1", it defines two custom Controls ("RAM_Usage" and "NET_Usage") based upon existing Skins and Functions and a ControlSet ("[Plugin] Sample1") that makes the new Controls available to the user.

It consists of two files:

plugin.json

```
{
  Name:"Sample1",
  CtrlSets:["Sample.ctrlset.json"],
  Controls:[
    {
      Name:"RAM_Usage",
      Skins:["RAM_Usage"],
      Functions:["RAM_Usage"],
      Config:{
        Description:"This is my Ram Usage!",
      },
      Width:1,
      Height:1,
    },
    {
      Name:"NET_Usage",
      Skins:["NET_Usage"],
      Functions:["NET_Usage"],
      Config:{
        Description:"This is my Network Usage!\nMultiline and stuff...",
      },
      Width:4,
      Height:1,
    },
  ],
}
```

Sample.ctrlset.json

```
{
  Name:"[Plugin] Sample1",
  Type:"ControlSet",
  ExpandCategories:["*"],
  Elements:[
    {
      Name:"Memory Usage",
      Control:"Sample1:RAM_Usage",
      Category:"Memory",
    },
    {
      Name:"Network Utilization",
      Control:"Sample1:NET_Usage",
      Category:"Network",
    },
  ],
}
```

Appendix A: Colors

Colors can be specified either by name or in one of multiple supported hexadecimal notations.

Hexadecimal notation

There are four supported notations for colors in the Power-Grid.

- "#AARRGGBB"
- "#RRGGBB"
- "#ARGB"
- "#RGB"

Named colors

In the following you can find the available color names and their associated hexadecimal color value.

- "White"   (#ffffff)
- "Black"   (#000000)
- "Red"   (#ff0000)
- "DarkRed"   (#800000)
- "Green"   (#00ff00)
- "DarkGreen"   (#008000)
- "Blue"   (#0000ff)
- "DarkBlue"   (#000080)
- "Cyan"   (#00ffff)
- "DarkCyan"   (#008080)
- "Magenta"   (#ff00ff)
- "DarkMagenta" (#800080)
- "Yellow"   (#ffff00)
- "DarkYellow" (#808000)
- "Gray"   (#a0a0a4)
- "DarkGray"   (#808080)
- "LightGray"   (#c0c0c0)
- "Transparent" (#00000000)

Appendix B: Alignment

Alignment definitions in the Power-Grid define the relation of smaller content to its bigger container and are realized through a String describing the anticipated position.

The following Alignments are available:

"LeftTop" 9	"CenterTop" 10	"RightTop" 12
"LeftCenter" 17	"CenterCenter" 18	"RightCenter" 20
"LeftBottom" 33	"CenterBottom" 34	"RightBottom" 36

Appendix C: Config Options

Config Options are used to make Controls configurable for the user.

Each ConfigOption JSON Object has the following properties:

Property (Type)	Description	Example
Name (String)	Name of the Config Option. The Option will be accessible through the Identifier String "Namespace:Name".	`Name:"MyConfigOption",`
Type (String)	A String specifying the Type of the Config Option.	`Type:"Text",`
Value (String)	Default value of this Config Option.	`Value:"here",`
Title (String)	Title for this Config Option as to be shown in the Control Editor.	`Title:"Enter Text:",`
Encrypt (Boolean)	Set to "true" to store this Config Option's Value as an encrypted String in the config file.	`Encrypt:false,`
Layout (String)	A String that may contain any of the following Layout instructions: • "<hr>" – Add a separator line after this option in Control Editor UI • " " – Add extra space after this option in Control Editor UI	`Layout:"<center>",`

Additionally depending on the ConfigOption Type more properties become available.

Type: None (hidden)

Not shown in the Control Editor but can be read and set from JavaScript and Dynamic Libraries.

Type: Label (non-editable text)

Label-type Config Options are shown as a non-editable plain text.

Labels can be used to put headers or other kinds of separators into the Control Editor.

Property (Type)	Description	Example
Text (String)	The Label's Text.	`Text:"Some Text",`
Layout (String)	A String that may contain any of the following additional Layout instructions: • "<b>" – bold font • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<b><center>",`

Type: Text (textbox)

Text-type Config Options are shown as a simple textbox that accepts every input.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`
Param (String)	A semicolon-separated String that may contain any of the following additional parameters: • "multiline=X" – accept newlines in the input string and show X lines of text (limit is 16)	`Param:"multiline=5",`

Type: Password (textbox)

Password-type Config Options are shown as a single line textbox that hides its input as it is common for password fields. For Password-type Config Options the Value is stored in encrypted form by default.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • "<stretch=X>" – stretch font by X	`Layout:"<stretch=2>",`

Type: Check (a single checkbox)

Check-type Config Options are shown as a simple checkbox that the user can check or uncheck. An optional half-checked state can be set from JavaScript. A Value of "0" or "false" resembles the unchecked state, "1" the partially checked state and a Value of "2" or "true" means the checkbox is fully checked.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`

Type: Radio (set of radio buttons)

Radio-type Config Options are shown as a group of radio buttons of which only one can be selected at any time. Value represents the zero-based index of the radio button currently selected.

Property (Type)	Description	Example

Items
(Array of Strings)
	A list of Strings representing the Items available for this Config Option.	`Items:[`
		`  "first option",`
		`  "second option"`
		`],`

Type: File (file selector / open file dialog)

File-type Config Options are shown as a textbox that accepts every input next to a "Select File"-button that opens a select file dialog when clicked.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`

Type: Folder (folder selector / choose folder dialog)

Folder-type Config Options are shown as a textbox that accepts every input next to a "Select Folder"-button that opens a "Select Folder" dialog when clicked.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`

Type: Shortcut

Shortcut-type Config Options are shown as a textbox that accepts every input next to a "Record"-button that opens a "Record Shortcut" dialog when clicked.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`

Type: Drop (dropdown)

Drop-type Config Options are shown as a drop-down control. Value represents the zero-based index of Item currently selected.

Property (Type)	Description	Example

Items
(Array of Strings)
	A list of Strings representing the Items available for this Config Option. Drop-type Config Options support a number of special Items that are explained in more detail below. All special Items except for "<hr>" implicitly set the "editable" parameter.	`Items:[`
		`  "first option",`
		`  "second option",`
		`  "<hr>",`
		`  "<openfile>"`
		`],`

Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`
Param (String)	A semicolon-separated String that may contain any of the following additional parameters: • "verbatim" – Value becomes the selected Item's text instead of its index • "editable" – Users are allowed to enter their own Value (implies the "verbatim" parameter)	`Param:"verbatim",`

The following special Items are available:

Item	Description
"<hr>"	Displays a horizontal separator Line. This Item cannot be selected.
"<imagefile>"	Opens a "Select File" dialog for choosing a supported image file when selected.
"<soundfile>"	Opens a "Select File" dialog for choosing a supported sound file when selected.
"<openfile>"	Opens a "Select File" dialog when selected.
"<startapp>"	Opens a "Select File" dialog for selecting an executable when selected.
"<openurl>"	Opens a "Open URL" dialog for selecting an URL when selected.
"<shortcut>"	Opens a "Record Shortcut" dialog when selected.
"<macro>"	Opens a "Edit Macro" dialog when selected.

Type: Button

Button-type Config Options are shown as a simple button that can be clicked.

Property (Type)	Description	Example
Text (String)	The Buttons's Text.	`Text:"Some Text",`
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`
Param (String)	A semicolon-separated String that may contain any of the following additional parameters: • "type=X" – X must be either "text" or "image" • "image=X" – X specifies the Identifier String of the Gfx to use for this button • "toggle" – if specified the button is a toggle button	`Param: "type=image;image=Gfx",`

Type: Spinner

Spinner-type Config Options are shown as a track-bar where users can move the slider to select any Integer inside the specified range.

Property (Type)	Description	Example
Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`
Param (String)	A semicolon-separated String that may contain any of the following additional parameters: • "range=X" – X format: "min-max"	`Param:"range=0-10",`

Type: Group (group of ConfigOptions)

Group-type Config Options wrap all following Config Options up to the next Group-Type Option into a collapsible Group.

Property (Type)	Description	Example
Text (String)	The Groups's Label-Text.	`Text:"My Option Group",`

Type: SpinDrop

SpinDrop-type Config Options are basically a track-bar with an associated drop-down control that offers some special Items.

Property (Type)	Description	Example

Items
(Array of Strings)
	A list of Strings representing the Items available for this Config Option's drop-down control.	`Items:[`
		`  "first option",`
		`  "second option"`
		`],`

Layout (String)	A String that may contain any of the following additional Layout instructions: • " " – center alignment • "<stretch=X>" – stretch font by X	`Layout:"<center>",`
Param (String)	A semicolon-separated String that may contain any of the following additional parameters: • "range=X" – X format: "min-max" • "suffix=X" – append X to the value selected with the track bar.	`Param:` `"range=0-10;suffix=s",`

Type: TextList

TextList-type Config Options are shown as a list where Text-Items can be added and removed from.

Type: Align

Align-type Config Options are shown as a special alignment selector control that can be used to select one of the alignments mentioned in Appendix B: Alignment.

Type: Position

Position-type Config Options are shown as a special position configurator control that can be used to set a position.

Type: ColorPicker

ColorPicker-type Config Options are shown as a special color selector control that can be used to choose a color.